

Does Programming Language Matter? An Empirical Study of Fuzzing Bug Detection

Tatsuya Shirai[†], Olivier Nourry[‡], Yutaro Kashiwa[†], Kenji Fujiwara[§], Hajimu Iida[†]

[†]Nara Institute of Science and Technology, Japan

[‡]The University of Osaka, Japan

[§]Nara Women's University, Japan

{shirai.tatsuya.sp1,yutaro.kashiwa,iida}@naist.ac.jp

nourry@ist.osaka-u.ac.jp,kenjif@ics.nara-wu.ac.jp

Abstract

Fuzzing has become a popular technique for automatically detecting vulnerabilities and bugs by generating unexpected inputs. In recent years, the fuzzing process has been integrated into continuous integration workflows (*i.e.*, continuous fuzzing), enabling short and frequent testing cycles. Despite its widespread adoption, prior research has not examined whether the effectiveness of continuous fuzzing varies across programming languages.

This study conducts a large-scale cross-language analysis to examine how fuzzing bug characteristics and detection efficiency differ among languages. We analyze 61,444 fuzzing bugs and 999,248 builds from 559 OSS-Fuzz projects categorized by primary language. Our findings reveal that (i) C++ and Rust exhibit higher fuzzing bug detection frequencies, (ii) Rust and Python show low vulnerability ratios but tend to expose more critical vulnerabilities, (iii) crash types vary across languages and unreproducible bugs are more frequent in Go but rare in Rust, and (iv) Python attains higher patch coverage but suffers from longer time-to-detection. These results demonstrate that fuzzing behavior and effectiveness are strongly shaped by language design, providing insights for language-aware fuzzing strategies and tool development.

CCS Concepts

• **Software and its engineering** → **Software testing and debugging**; **Language types**; **Empirical software validation**; • **Security and privacy** → **Software security engineering**.

Keywords

Continuous Fuzzing, Programming Languages, OSS-Fuzz, Vulnerability Detection

ACM Reference Format:

Tatsuya Shirai[†], Olivier Nourry[‡], Yutaro Kashiwa[†], Kenji Fujiwara[§], Hajimu Iida[†]. 2026. Does Programming Language Matter? An Empirical Study of Fuzzing Bug Detection. In *23rd International Conference on Mining Software Repositories (MSR '26)*, April 13–14, 2026, Rio de Janeiro, Brazil. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3793302.3793358>

1 Introduction

Fuzzing is a testing methodology that automatically generates a large number of inputs and feeds them to a target software system to trigger unintended behaviors (*e.g.*, crashes or memory overflows) and uncover defects. The effectiveness of fuzzing has been demonstrated by multiple studies [1, 6].

To make fuzz testing more accessible in modern software development workflows, continuous fuzzing integrates fuzzing directly into continuous integration (CI) pipelines with frequent, short testing sessions. This approach addresses the traditional challenges of setting up complex fuzzers and managing high computational costs [25]. Rather than running for weeks on a single software version, continuous fuzzing operates daily on actively developed code. Klooster *et al.* [17] conducted a study on the effectiveness and scalability of fuzzing within CI pipelines and demonstrated that even 15-minute sessions can uncover critical bugs.

In recent years, several studies have investigated continuous fuzzing practices and software defects detected through continuous fuzzing (hereafter referred to as fuzzing bugs). Ding *et al.* [7] analyzed the characteristics of fuzzing bugs, finding that many of them cause the software to become unavailable to users, *e.g.*, by causing crashes or resource exhaustion, and that non-reproducible bugs are less likely to be fixed. Keller *et al.* [15] also conducted a lifecycle analysis of fuzzing bugs, revealing that while the median lifespan of fuzzing bugs was 324 days, they were addressed within a median of only 2 days after detection.

While these prior studies have provided interesting new insights on continuous fuzzing, their analyses do not distinguish between programming languages or focus on specific languages. Programming languages vary significantly in their design principles and safety guarantees. For example, Rust emphasizes memory safety through compile-time checks, while others like C/C++ provide low-level memory control at the cost of safety. Given these fundamental differences, it is still not clear whether prior findings, derived from language-agnostic analyses, can be directly generalized across multiple languages. In this context, we thus pose the central question:

“Do fuzzing sessions/bugs manifest differently across languages?”

In this study, we perform a cross-language analysis to investigate how fuzzing bug characteristics and detection efficiency vary across programming languages. We analyze 61,444 fuzzing bugs, and 999,248 builds from 559 OSS-Fuzz repositories, categorized by primary language. Our contributions are:



This work is licensed under a Creative Commons Attribution 4.0 International License. *MSR '26, Rio de Janeiro, Brazil*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 978-1-4503-XXXX-X/18/06

<https://doi.org/10.1145/3793302.3793358>

- (1) **Programming Languages show distinct bug detection Distributions:** Although the median bug detection rate is generally low across all languages (around 0.02 issues per build), the frequency distribution differs significantly across languages. Specifically, C++ and Rust exhibit the widest distributions and the highest variability, with long tails extending far beyond the median, while Python shows the lowest median detection rate (0.014) and the narrowest, most concentrated pattern.
- (2) **Language differences shape vulnerability patterns:** The nature of vulnerabilities differed sharply across languages. Specifically, Java projects exhibited a distinct profile dominated by medium-severity (S3) vulnerabilities, contrasting with the predominance of critical and high-severity (S1/S2) vulnerabilities in C, C++, Go, Python, and Rust. Furthermore, C, C++, and Go exhibit mainly Resource Management bugs, whereas Java, Python, and Rust tend to show more Control Flow bugs.
- (3) **Rust demonstrates superior fuzzing bug reproducibility:** Rust projects demonstrate superior fuzzing bug reproducibility. These projects showed the best reproducibility ratio of fuzzing bugs (median around 98.8%) and a tight distribution near 100%. Additionally, while Rust projects detect numerous fuzzing bugs, their vulnerability ratio remains low (below 10%).
- (4) **Fuzzing efficiency depends on language characteristics:** Fuzzing efficiency, measured by Patch Coverage and Time-to-Detection (TTD), varies significantly across languages. Interestingly, languages with relatively high patch coverage, such as Go and Python, tend to show longer time to detection, while C, C++, Java, and Rust exhibit lower patch coverage but shorter detection times. This suggests that the expected relationship where testing newly modified code (high patch coverage) facilitates early bug detection does not universally hold across all programming languages when using fuzzing.
- (5) **Replication Packages:** To facilitate replication and further studies, we provide the data used in our replication package.¹

2 Background

2.1 Continuous Fuzzing

Continuous fuzzing integrates the fuzzing process directly into the CI pipeline, running automated fuzz tests regularly (*e.g.*, daily). While traditional fuzzing is typically performed manually (*e.g.*, before releasing), continuous fuzzing executes in short, frequent cycles. This approach enables teams to quickly and automatically detect bugs as soon as they enter the codebase, rather than discovering them later during development.

OSS-Fuzz, provided by Google, is the most widely used continuous fuzzing service, currently integrating and fuzzing over 1,000 critical open-source software projects. Since its launch in 2016, OSS-Fuzz has helped identify and fix over 13,000 vulnerabilities and 50,000 bugs across participating projects. It officially supports six languages, including C, C++, Rust, Go, Python, Java, and JavaScript,² enabling fuzzing across many critical projects such as OpenSSL, Kubernetes, and TensorFlow.

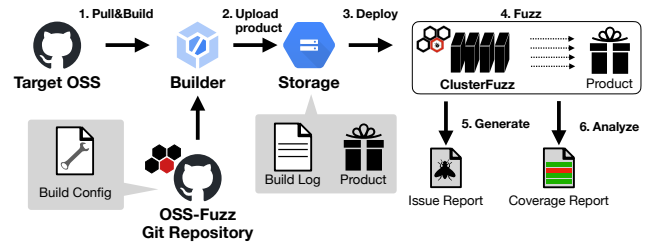


Figure 1: Architecture of OSS-Fuzz

Figure 1 illustrates the bug detection process using OSS-Fuzz. To use the service, developers must ensure that their repository is public, must write their own fuzz targets, and have to upload their build configuration files to OSS-Fuzz. Using these build files, OSS-Fuzz will then be able to build the target open-source project and its fuzz targets to carry out the fuzzing process.

After the build completes, a fuzzing build log is generated and uploaded to a Google Cloud Storage (GCS) bucket.³ The compiled project files and fuzz targets are also uploaded to cloud storage for use by OSS-Fuzz’s fuzzers.

Next, ClusterFuzz, the distributed fuzzing infrastructure underlying OSS-Fuzz, downloads the compiled source files along with the initial fuzzing inputs (*i.e.*, the fuzzing corpus) and begins fuzzing. By default, ClusterFuzz uses libfuzzer⁴ as its fuzzer, and more than 99% of projects in OSS-Fuzz use either libfuzzer by itself or in combination with other fuzzers. When ClusterFuzz’s fuzzers trigger a bug during fuzzing, an issue report is automatically generated and submitted to the issue tracking system.⁵

Each issue report includes the project name, the fuzzing engine used during the execution, the crash information, the severity of the bug, the regression range (*i.e.*, the range of commits where the bug was introduced), and the fixed range (*i.e.*, the range of commits where the bug was fixed). Reports remain private for 90 days from the reporting date or until the bug is fixed by the projects.

Additionally, once the fuzzing process is completed, a daily coverage report is generated which provides various fuzzing coverage information such as the hit frequency, the fuzzer reachability reports, and the aggregated coverage reports to understand the performance of the fuzzer and identify any blockers [9]. These coverage reports are also uploaded to the OSS-Fuzz GCS bucket and made available publicly.⁶

2.2 Related Work

We categorize related work into three approaches: language-agnostic studies (no differentiation between languages), language-specific studies (analyzing fuzzing practices and bugs for a particular language), and cross-language studies (examining specific bug types across different languages).

³<https://oss-fuzz-build-logs.storage.googleapis.com/log-61e82104-9c63-41eb-83f6-6b0bfcf861e3.txt>

⁴<https://lvm.org/docs/LibFuzzer.html>

⁵<https://issues.oss-fuzz.com/issues?q=status:closed>

⁶<https://storage.googleapis.com/oss-fuzz-coverage/openssl/reports/20250613/linux/report.html>

¹<https://doi.org/10.5281/zenodo.18264483>

²<https://github.com/google/oss-fuzz>

2.2.1 Language-Agnostic Analysis. Many studies have examined the effectiveness of continuous fuzzing [15], the characteristics of detected bugs [7, 43], and the effectiveness and limitations of fuzzing techniques [3, 16, 17, 21]. While these studies provided valuable overall insights into fuzzing, continuous fuzzing, and fuzzing bugs, they have not considered the influence of programming languages, leaving language-specific characteristics unexplored.

Zhu *et al.* [43] analyzed approximately 23,000 bugs reported in OSS-Fuzz and found that about 77% of them were introduced by recent code changes. They observed that the proportion of these regression bugs increased in projects that had been fuzzed continuously over long periods. Ding *et al.* [7] analyzed 23,907 bugs detected by OSS-Fuzz to investigate their characteristics (such as bug types, severity, and flakiness). They found that 52% of the bugs primarily harmed system availability, such as crashes, timeouts, and out-of-memory errors. The median time from introduction to detection and from detection to fix is 5 days and 5.3 days, respectively. Keller *et al.* [15] studied 44,102 OSS-Fuzz issues and their corresponding commits. They found that bug discovery rates decreased over time in most projects and that approximately 46% of bugs were fixed by the same developers who introduced them.

Nourry *et al.* [26] investigated the issue of build failures in fuzzing and reported that approximately 9.4% of OSS-Fuzz projects experience build failures without distinguishing between languages. Shirai *et al.* [33] studied the evolution of fuzzing coverage, using 1 million fuzzing sessions from 808 projects on OSS-Fuzz. They showed that code coverage tends to increase continuously and that bug detection rates rise during coverage fluctuations.

2.2.2 Language-Specific Analysis. Several studies have conducted extensive analyses on various aspects of fuzzing itself (*e.g.*, coverage, efficacy, evolution) in a specific language or several related languages such as C and C++ [10].

Liyanage *et al.* [21] analyzed reachable coverage during long-term fuzzing using 32 real-world C programs. They observed that branch coverage increased linearly on a logarithmic time scale. Böhme *et al.* [3] analyzed the reliability of coverage-based fuzzer benchmarking on a large variety of widely-used open-source C programs from different domains. They revealed that there is a strong correlation between the coverage achieved and the number of bugs found by a fuzzer. Nourry *et al.* [27] analyzed the adoption and evolution of fuzzing in 11,341 Go projects. Their results showed that the adoption of fuzzing remained limited, being used in only 3.15% of all testing functions.

Hassler *et al.* [10] analyzed over 100 vulnerabilities injected into C/C++ programs to evaluate 13 fuzzers. They revealed that fuzzing for C/C++ is most effective in detecting vulnerability types related to memory corruption and resource management errors, whereas it is less effective in identifying numerical computation errors. Klooster *et al.* [17] studied the practical effectiveness of fuzzing and showed that even short fuzzing sessions of 10-30 minutes can achieve comparable detection effectiveness to long-running ones.

Much of the prior work has also focused their analysis on the kinds of bugs that tend to get introduced into specific languages. These studies are first and foremost language-centric, meaning that they focus on the characteristics of the language unlike cross-language analysis studies which focus on bug characteristics.

For example, Imtiaz *et al.* [13] analyzed vulnerabilities detected in C and C++ projects. Their results showed that approximately 40% of the vulnerabilities were memory-related, and that the average CVSS3 score (*i.e.*, a measure of vulnerability severity) was 9.3 for memory-related vulnerabilities, which is higher than 6.6 for non-memory ones. Zhang *et al.* [41] analyzed 790 bugs from Rust projects and identified their root causes. They found that only 0.9% of these bugs were security vulnerabilities, 53% were caused by algorithm implementation errors, and only 3.8% were related to memory issues. Oh *et al.* [28] analyzed errors in real-world Python applications. Their results showed that type error exceptions accounted for about 30% of all cases and were the most frequent, and that approximately 30% of these required more than one month to fix, indicating a tendency toward prolonged resolution.

2.2.3 Cross-Language Analysis. While fuzzing research has yet to compare bugs across languages, many studies conduct bug-centric analyses to investigate how specific kinds of bugs manifest in different languages, how bug-fixes are implemented in different languages, or try to characterize vulnerabilities and bugs across multiple languages [18, 23, 24, 32].

Nikitopoulos *et al.* [24] constructed a multilingual CVE dataset and organized the collected CVEs based on CWE categories. Their analysis showed that CWE-119 (*i.e.*, Bounds of a Memory Buffer) was the most common category in C language files, whereas CWE-79 (*i.e.*, Cross-site Scripting) was the most prevalent in PHP files. Ray *et al.* [32] analyzed the number of defect-fixing commits across 17 programming languages and 729 projects. They showed that although the influence of programming languages on the number of defect fixes was minor, there was a strong correlation between languages and specific bug types, such as memory errors and concurrency issues. Spinellis *et al.* [34] investigated the behavior of fuzzing across programming languages in response to small changes in source code. Their results show that in strongly typed systems, program changes can be detected at compile time, whereas in weakly typed systems, the program may still execute and produce incorrect outputs. Kochhar *et al.* [18] investigated the bug trends in projects developed using multiple programming languages. They found that using multiple languages increases both the number and diversity of bugs. Nanz and Furia [23] compared solutions to 745 programming tasks from Rosetta Code, a multi-language code repository. They examined the fault-proneness of different languages and found that Go was the least fault-prone, whereas Python and Ruby tended to be the most fault-prone.

2.2.4 Summing Up. Related work indicates that fuzzing research primarily targets single-language environments (*e.g.*, C, C++, or Go). In contrast, vulnerability analyses frequently compare across multiple languages, establishing a correlation between language choice and distinct bug patterns. However, most large-scale empirical studies on continuous fuzzing have been language-agnostic. Given the fundamental differences in language design and safety guarantees—such as Rust emphasizing memory safety versus C/C++ providing low-level control—it remains uncertain whether general fuzzing effectiveness and findings apply across all languages. This ambiguity highlights the necessity for a comprehensive cross-language analysis of how fuzzing characteristics and bug detection efficiency are shaped by the programming language used.

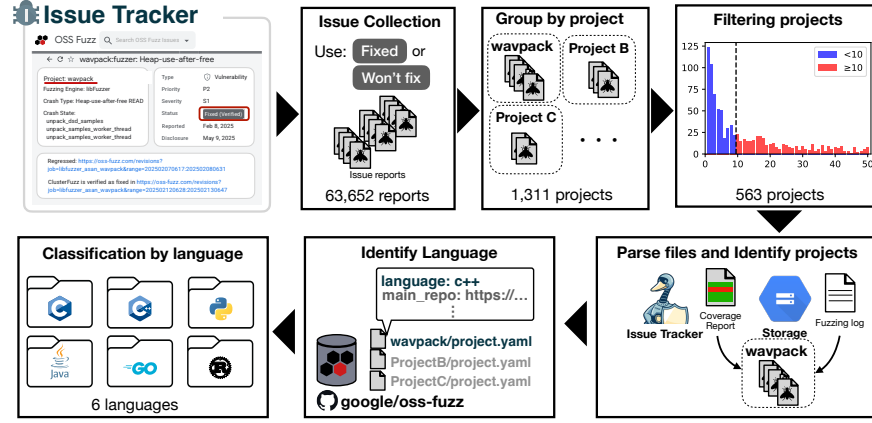


Figure 2: Overview of the data collection

3 Data Collection

This section describes the procedures for data collection. Figure 2 depicts the overview of the data collection process.

3.1 Issue Reports

This study examines fuzzing bugs (*i.e.*, bugs discovered during fuzzing sessions). We collect issue reports from OSS-Fuzz’s issue tracker.⁷ Specifically, we collect all issue reports submitted by 2025-06-15, totaling 68,495 reports. We then filter for reports with status “Fixed” or “Won’t fix” (*i.e.*, non-Open issues) to focus on conclusively resolved issues, yielding 63,652 reports.

Next, we select appropriate projects to be analyzed. As of September 2025, 1,311 OSS projects are continuously fuzzed by OSS-Fuzz. However, some projects lacked sufficient issue reports for reasons such as having only recently begun fuzzing. Given the continuous operation of OSS-Fuzz, finding almost no bugs over an extended period is improbable and likely indicates misconfigurations or ineffective fuzz targets (*e.g.*, shallow code coverage) rather than the absence of bugs. Because our analysis computes median values for projects grouped by programming language, we selected only projects with at least ten detected fuzzing bugs. This results in 563 projects (42.9 % of the total).

3.2 Fuzzing Build Logs and Coverage Reports

In RQ1, we use the number of sessions performed by OSS-Fuzz to calculate the average number of fuzzing bugs found by fuzzing sessions. To measure it, we first collect all the OSS-Fuzz build logs (4,660,639 logs) for each project by scraping from Google Cloud Storage of OSS-Fuzz.⁸ We then parsed the log files, identified the projects, and filtered logs only for the studied repositories. In the end, we collected 999,248 build logs from the studied repositories.

Additionally, we use coverage reports in RQ4, which show the extent to which the fuzzing sessions explore the code base. These coverage reports in OSS-Fuzz are generated daily (Note: not after every revision). We obtained 669,015 coverage reports in total.

⁷<https://issues.oss-fuzz.com/issues>

⁸We needed to collect all because the information in each build log (*e.g.*, the build type and the target project) cannot be identified without parsing it.

3.3 Classification by Language

We identified the primary language used in each project and categorized the issues accordingly. We determined the main language using project descriptions from the OSS-Fuzz GitHub repository.⁹ For the 10 projects where the configured language was changed, the change was likely a correction from the default setting to the project’s main language or to add support for an additional language. After grouping by language, we found insufficient numbers of Swift and JavaScript projects (2 and 2 projects, respectively) and thus excluded them from our analysis. Table 1 presents summary statistics of the projects used in this study. It is worth noting that most of the issues were discovered by libfuzzer, which is the default fuzzer used by OSS-Fuzz. Specifically, libfuzzer detected 88.6%-100% of issues in C, Go, Java, Python, and Rust, versus 58.9% in C++ (32% of C++ reports do not mention which fuzzer was used).

4 Research Questions

RQ1: Does the frequency of bugs detected through continuous fuzzing differ by language?

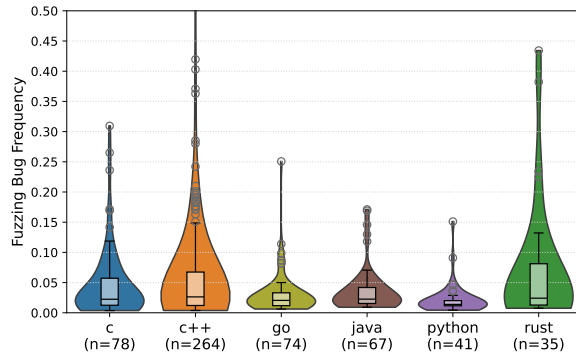
Motivation. Shirai *et al.* [33] investigated the frequency of bugs discovered through continuous fuzzing, revealing that the bug detection rate fluctuates around 5%. However, their study did not distinguish which languages were used in the studied projects. Different results may be obtained when examining modern software ecosystems that rely on diverse programming languages with varying memory management paradigms, type systems, and security features. To address this gap, we examine the frequency of fuzzing bugs in different programming languages.

Approach. We investigate how frequently fuzzing sessions find bugs. To measure the bug detection rate, we count the number of bugs found in OSS-Fuzz and divide by the total number of fuzzing builds. We repeat this calculation for each project, aggregate the results by programming language, and calculate the median value for each language. To check if different languages have significantly different detection rates, we apply the Kruskal–Wallis test

⁹<https://github.com/google/oss-fuzz/blob/master/projects/abseil-cpp/project.yaml>

Table 1: Summary of studied projects and collected data

Language	# Projects	# Issues	# Build Logs	# Coverage Reports	Median Commits	Median LOC
C	78	5,364	139,759	89,064	6,151	199,011
C++	264	47,128	591,651	451,767	4,578	126,216
Go	74	3,180	109,425	20,349	5,451	133,691
Java	67	2,339	66,608	47,101	3,239	77,558
Python	41	769	44,750	30,236	1,397	25,602
Rust	35	2,664	47,055	30,498	3,924	79,096
Total	559	61,444	999,248	669,015	–	–

**Figure 3: Detection frequency of all fuzzing bugs relative to fuzzing runs.**

($\alpha = 0.05$) [19]. The Kruskal–Wallis test is a non-parametric statistical method that assesses whether samples originate from the same distribution, without assuming normality or equal variances. Since the Kruskal–Wallis test can only detect whether differences exist among the distributions, we apply the Mann–Whitney U test [22] with Holm correction [11] ($\alpha=0.05$) as a post-hoc test to identify which pairs of distributions differ significantly. Note that we apply this same statistical procedure (Kruskal–Wallis test followed by Mann–Whitney U test with Holm correction, $\alpha=0.05$) throughout RQ1–RQ4 whenever comparing distributions across different categories.

Results. C, Go, and Java show similar median bug detection rates. C++ and Rust exhibit higher median bug detection with wider spread, whereas Python shows the lowest median and narrowest spread. Figure 3 shows the distribution of total fuzzing bugs per build across six programming languages. We observe that the overall detection rates are low in all languages, with median bug detection rates around 0.02 fuzzing bug per build, as indicated by the horizontal lines in the box plots.

When examining the median values for each language, we observe subtle differences. First, the median values for C, Go, and Java are remarkably similar (approximately 0.02 fuzzing bugs per build), while C++ and Rust exhibit slightly higher medians (0.025), and Python show slightly lower medians (0.014). Despite similar median values, the languages differ significantly in their variability. C++ and Rust exhibit the widest distributions with long tails extending up to 0.15–0.45 fuzzing bug per build. C and Java show

moderate variability, while Go displays a slightly more compact distribution. Python demonstrates the least variability with a narrow distribution centered near zero.

A Kruskal–Wallis test confirmed statistically significant differences across languages ($H = 14.28$, $p = 0.014$), indicating that fuzzing bug detection rates vary by language. With the post-hoc tests to examine the difference between each pair, we observed statistically significant differences between C++ and Python, and between Java and Python.

Answer to RQ1: While C, C++, Go, and Rust show similar median fuzzing bug detection rates (0.02 fuzzing bugs per build), they differ substantially in variability, with C++ and Rust exhibiting the widest distributions and Python showing the most concentrated pattern with the lowest overall rate.

RQ2: Do fuzzing bug types and severity differ by language?

Motivation. Previous studies [7, 10, 15] have investigated the characteristics of fuzzing bugs. Hassler *et al.* [10] compared fuzzers and static analysis tools, revealing fuzzers are particularly effective at detecting memory-related vulnerabilities such as bounds violations and use-after-free errors. However, these findings may not apply to memory-safe languages like Rust, where compile-time safety guarantees prevent entire classes of those vulnerabilities. The fundamental differences in memory management between C and Rust likely result in distinct fuzzing bug patterns. While C may reveal more memory corruption issues, Rust would be expected to detect logic errors or unsafe block vulnerabilities. Even if memory-related vulnerabilities are detected in Rust, the severity may be mitigated by its safety mechanisms. Therefore, RQ2 examines OSS-Fuzz data across multiple languages to understand how language design influences vulnerability discovery rates.

Approach. We measure three metrics: Vulnerability Ratio, Severity Level, and Crash Type, as follows.

i) *Vulnerability Ratio* : Once a defect is detected by the fuzzers, OSS-Fuzz then classifies it into one of two types: “Bug” and “Vulnerability”. “Bug” indicates an unexpected or incorrect program behavior and “Vulnerability” refers to a privacy or security issue as defined by Google’s security guidelines.¹⁰ Using the Type field found in issue reports, we calculate the distribution of each type across all fuzzing bugs, grouped by language.

¹⁰<https://developers.google.com/issue-tracker/concepts/issues#type>

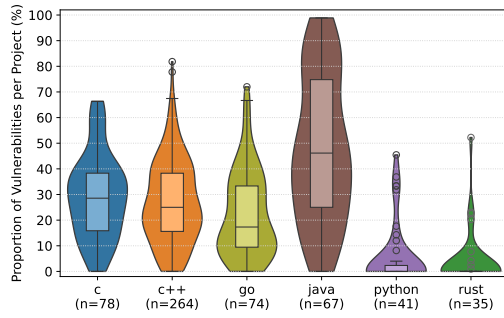


Figure 4: Proportion of fuzzing bugs with Vulnerability label.

ii) *Severity Level* : OSS-Fuzz automatically¹¹ assigns severity levels to crashes when creating issue reports. Each issue report contains a *Severity* field that ranges from “S1” (most severe) to “S4” (least severe). We measure the distribution of severity levels for each language group. It is worth noting that most of the issues with “Bug” tag have “S4” level so we only study the severity of issues with the “Vulnerability” tag.

iii) *Crash Type* : Fuzzing bugs can be classified into multiple types based on their CWE-ID [7, 35]. Issue reports in OSS-Fuzz contain a *Crash Type* field that reveals how crashes manifest. We classify them, employing the highest-level categories (*i.e.*, Pillar categories) from the CWE VIEW: Research Concepts.¹² Table 2 lists the CWE-IDs and their corresponding names used in this study. We calculate the distribution of CWE categories across all fuzzing bugs grouped by programming languages.

Results. Vulnerability rates are lowest in Python and Rust, higher in C and C++, while Java has the highest rate overall. Figure 4 shows the vulnerability ratio among fuzzing bugs across different languages. With an overall vulnerability rate of 23.48 %, C and C++ exhibit ratios close to the overall rate. These languages demonstrate not only high occurrences of fuzzing bugs (as shown in RQ1) but also a substantial number of vulnerabilities. Notably, Java projects have the highest vulnerability ratio, with nearly half (46.3%) of their fuzzing bugs classified as vulnerabilities. In contrast, Python and Rust have a significantly lower proportion of vulnerabilities. Notably, while RQ1 revealed that Rust projects detect numerous fuzzing bugs, their vulnerability ratio remains below 10%. We observed statistically significant differences detected by Kruskal-Wallis tests and also between each pair, except C and C++ as well as Rust and Python.

Severity varies dramatically by language, with Java showing almost exclusively medium-severity vulnerabilities (S3) while all other languages are dominated by critical and high-severity vulnerabilities (S1-S2). Figure 5 shows the severity distribution of vulnerability-labeled fuzzing bugs across six programming languages. We observe several different tendencies by languages. The most striking pattern emerges with Java, which exhibits an entirely unique profile where nearly 100% of its vulnerability-labeled fuzzing bugs fall into the S3 (medium severity) category.

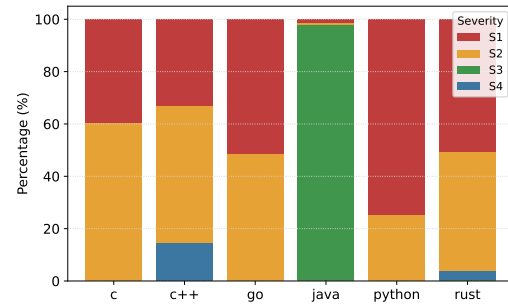


Figure 5: Severity (S1–S4) distribution by language for Vulnerability-labeled fuzzing bugs.

This unique pattern may be attributed to Java’s memory-managed nature and built-in safety features that prevent many of the critical vulnerabilities found in other languages [29].

In contrast to Java, most other languages are dominated by high-severity vulnerabilities. C, C++, Go, and Rust show predominantly S1 (highest severity) and S2 vulnerabilities, with S1 typically accounting for 30-50% of the vulnerabilities discovered through fuzzing. Python also exhibits an exceptionally high proportion of S1 vulnerabilities at approximately 75%, markedly exceeding the rates observed in other languages.

The results also reveal that S4 (lowest severity) vulnerabilities are quite rare across all languages. Only C, C++ and Rust show any S4 vulnerabilities at all, and even in these cases, they represent a very small percentage, less than 20% of the total.

Memory-unsafe languages (C, C++) are dominated by Resource Management bugs, whereas managed languages (Java, Python) primarily suffer from Control Flow-related bugs. Figure 6 shows the distribution of crash types across programming languages as a heatmap, with darker red indicating higher values.

Memory-unsafe languages exhibit distinct patterns. C and C++ show the highest Resource Management bug rates at 48.6% and 50.7%, respectively, reflecting their manual memory management requirements. In contrast, Java, Python, and Rust are predominantly affected by Control Flow bugs. This trend is most pronounced in Python, where Control Flow bugs account for 79.0% of all fuzzing bug. Java also shows 50.2% Control Flow bugs and is notably the only language exhibiting Protection Mechanism bugs. Rust demonstrates interesting characteristics. Although Control Flow bugs are dominant at 47.0%, Resource Management bugs also account for 26.4%. Since this is significantly lower than in memory-unsafe languages, Rust’s ownership system helps prevent resource-related crashes but does not fully eliminate logic-level vulnerabilities. Go, on the other hand, shows a more balanced distribution of bug types, without strong concentration in any single category.

Answer to RQ2: Java projects have the most vulnerabilities but mainly medium-severity ones, while Python and Rust have fewer but more critical issues. C, C++, and Go show similar patterns. For crash types, C, C++, and Go mainly have Resource Management bugs, while Java, Python, and Rust have Control Flow bugs.

¹¹https://github.com/google/clusterfuzz/blob/5b7f6aa8daf56b3f33508855f17fe77570baf19a/src/clusterfuzz/_internal/crash_analysis/severity_analyzer.py

¹²<https://cwe.mitre.org/data/definitions/1000.html>

Table 2: Types of CWE

Label	CWE-ID	Description	Example Crash Type
Resource Management	CWE-664	Improper Control of a Resource Through its Lifetime	Timeout, Out-of-memory, buffer-overflow
Incorrect Calculation	CWE-682	Incorrect Calculation	integer-overflow, divide-by-zero
Control Flow	CWE-691	Insufficient Control Flow Management	assert, stack-overflow, ill
Protection Mechanism	CWE-693	Protection Mechanism Failure	security-exception,
Exceptional Handling	CWE-703	Improper Check or Handling of Exceptional Conditions	abrt, unexpected-exit, fatal-signal
Neutralization	CWE-707	Improper Neutralization	index-out-of-range, slice-bounds-out-of-range
Coding Standards	CWE-710	Improper Adherence to Coding Standards	null-dereference, undefined-shift

Note: We distinguish stack-overflow from buffer-overflow based on their root causes (i.e., control recursion vs. memory bounds).

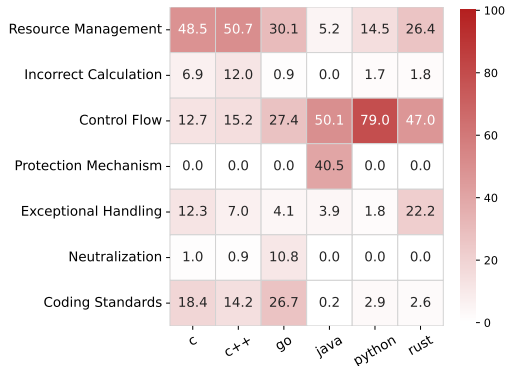


Figure 6: Proportion of all fuzzing bugs by crash types across languages.

RQ3: Does the reproducibility of discovered fuzzing bugs differ by language?

Motivation. Reproducing bugs consistently is important for developers to fix them. To facilitate bug fixing, fuzzing infrastructure like ClusterFuzz provides the necessary information to reproduce issues and confirm that fixes are correct. However, reproducibility is not always guaranteed [8]. Ding *et al.* [7] showed that 13% of reported issues in OSS-Fuzz are unreproducible, which can hinder the bug-fixing process and waste developer effort. While their study provides valuable insights into the overall reproducibility problem, they did not distinguish between different programming languages. Given that different languages have varying memory management models, runtime behaviors, and debugging toolchains, reproducibility rates may differ significantly across languages.

Approach. OSS-Fuzz classifies each crash as either “Reproducible” or “Unreproducible” in its reports. Unreproducible crashes occur intermittently due to factors like race conditions or uninitialized memory. We calculate the proportion of “Reproducible” bugs for each studied language.

Results. *Reproducibility varies across programming languages, with Go showing the lowest reproducibility rate (median about 50%), and C/C++/Python/Java performing better (80-95%).* Figure 7 displays the distribution of reproducibility ratio per language using violin plots with embedded box plots. Looking at the overall results, there are significant variations in reproducibility rates across the different programming languages. The majority

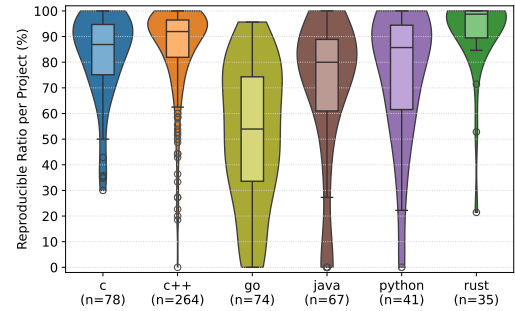


Figure 7: Reproducibility rate of crashes across languages.

of projects have relatively high reproducibility ratios, but some outliers reach very low percentages. Go shows the lowest reproducibility, with a median reproducibility ratio of about 50% and a broad distribution that extends across nearly the entire range. C, Java, and Python exhibit similar patterns with medians around 85% of reproducible crashes and fairly concentrated distributions, suggesting comparable reproducibility challenges. C++ presents an interesting case with a high median around 90% but notable outliers reaching as low as 10%, indicating that while most C++ projects are reproducible, some face significant challenges.

Rust projects demonstrate the best reproducibility characteristics with the highest median around 98.8% and a narrow distribution, indicating that almost all fuzzing bugs are consistently reproducible. Among the languages studied, Rust projects demonstrate exceptional fuzzing bug reproducibility with the highest observed reproducibility ratio (median 98.8%) and the most concentrated distribution near 100% among all studied languages. This superior performance suggests that Rust’s design features and toolchain effectively promote deterministic and consistent behavior compared to other programming languages. We observed statistically significant differences detected by the Kruskal-Wallis test. Specifically in post-hoc tests, we found significant differences ($p < 0.05$) between all language pairs, except for C vs. C++, C vs. Python, and Java vs. Python.

Answer to RQ3: Rust projects demonstrate the best reproducibility with a median ratio of reproducible crashes around 98.8% and a tight distribution near 100%, while Go projects perform worst with a median ratio near 50% and a broad distribution.

RQ₄: Does fuzzing efficiency differ by language?

Motivation. Coverage-Guided Fuzzing (CGF) is the primary fuzzing strategy adopted by OSS-Fuzz, leveraging knowledge of the code-base and coverage information to explore a broader range of code paths. Higher code coverage is generally regarded as an indicator of fuzzing effectiveness for a given software system. Shirai *et al.* [33] reported that improving coverage is more likely to help detect bugs, demonstrating a direct relationship between coverage and bug discovery. Another critical metric for evaluating fuzzing efficiency is Time-to-Detect, which refers to the time elapsed between a bug’s introduction and detection. Ding *et al.* [7] showed that OSS-Fuzz detects the majority of identified regressions within a week, highlighting the practical value of rapid bug detection. However, it remains unclear whether fuzzing efficiency varies across programming languages.

Approach. We measure two proxy metrics for fuzzing efficiency as follows.

i) *Patch Coverage* : Patch coverage measures how much of the modified code in commits is exercised by test suites [40]. Fuzzers for continuous fuzzing are often evaluated based on how effectively they reach the newly modified code [4, 38, 42]. To calculate this, we first identified the revisions and dates on which each target project was changed, based on the daily coverage reports. We then determined the modified code (the diff) for each of those revisions. Finally, we matched the coverage reports against the diff-covered code regions in order to compute patch coverage. We then calculate patch coverage by matching the modified code regions with the corresponding coverage reports. To provide representative median values, we only selected projects for which patch coverage could be computed for at least 10 revisions in each project (312 projects). Finally, we iteratively apply this process for each project, compute the median patch coverage, and aggregate the results by programming language to analyze patch coverage distributions.

ii) *Time To Detection* : Time-to-detection (TTD) is the number of days between when fuzzing bugs are introduced and when they are detected by fuzzing. Similar to prior work by Ding *et al.* [7], we identified the introduction date of fuzzing bugs by checking the “Regressed Revision” field in issue reports, which indicates the build in which the bug first appeared. We marked the detection date as when the issue report was filed, then calculated the TTD as the days between these two dates. We compute the median TTD for each project and examine the distributions of these medians for each programming language.

Note, since this analysis relies on the availability of the “Regressed Revision” field of the issue report, we excluded issues where this information could not be extracted. We further filtered the projects that have fewer than 10 issue reports to obtain representative median values. As a result, this analysis targeted 345 projects and 43,002 issues in total. It is worth noting that all reports with the “vulnerability” tag in Go projects lacked the “Regressed Revision” entry. To keep our data consistent across languages, we therefore had no choice but to exclude GO projects from this analysis.

Results. *Patch coverage varies significantly across languages. Go and python show the highest coverage, with a median hovering around 30.2% and 22.9% respectively. In contrast,*

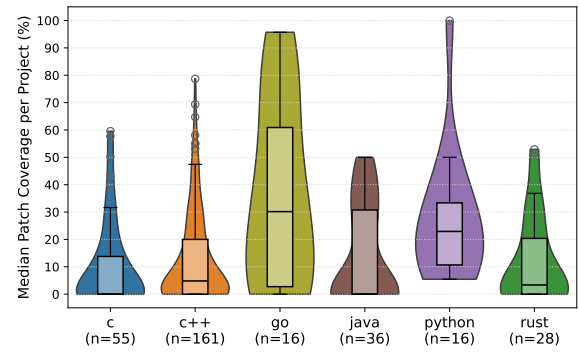


Figure 8: Distributions of Patch Coverage across languages.

C, C++, Java, and Rust all have medians below 10%, though some projects exceed 50%. Figure 8 shows the distribution of per-project patch coverage for each programming language. The results indicate a clear distinction between two groups: languages that maintain relatively high patch coverage (Go and Python) and those that exhibit consistently low coverage (C, C++, Java, and Rust). Notably, Go shows a relatively higher median patch coverage around 30.2%, likely due to its built-in fuzzing support and simple language design. Similarly, Python achieves a comparatively high median of around 22.9%. As the only dynamically typed language in our study, Python’s runtime flexibility may allow fuzzers to explore diverse code paths more easily, though further investigation is needed to confirm this hypothesis. While the Kruskal-Wallis test detects overall differences among distributions, post-hoc tests identify significant differences between Python and both C and C++, as well as between C and Go.

In contrast, several programming languages (C, C++, and Rust) show wide distributions, but their median patch coverage remains below 10%. This may be because of the high structural complexity of these languages which hinders fuzzers’ ability to achieve high coverage. This could also be due to the high amount of fuzzing bugs (as shown by RQ1) in these languages preventing subsequent code execution (*i.e.*, the code execution is terminated by crashes).

Time-to-detection (TTD) varies substantially across programming languages. C and C++ tend to detect bugs within about a week, whereas Java and Rust typically detect them within around three days. In contrast, Python shows a tendency toward much longer detection times. Figure 9 presents the distributions of median TTD time to detect fuzzing bugs across projects, grouped by programming language.

The C and C++ programming languages exhibit similar distributions and median values (7.0 and 9.5 days, respectively), indicating that bugs in these languages are typically detected by fuzzing within roughly one week. As shown in RQ2, these languages often involve memory-related issues that require complex inputs, which might contribute to longer detection times.

In contrast, Java and Rust show much shorter median TTDs of 2.8 and 3.9 days, respectively, indicating that bugs are detected more quickly. These languages tend to have a higher proportion of control-flow-related bugs in common. Similarly, Python projects

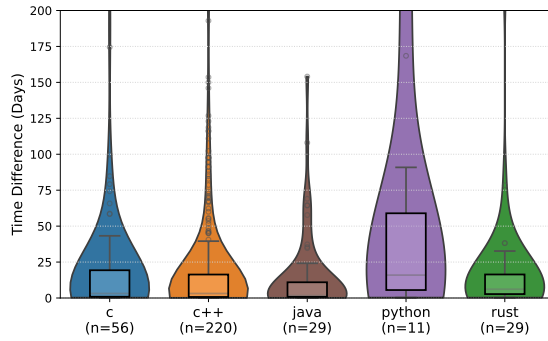


Figure 9: Distributions in the median time to detect fuzzing bugs discovered by each project, grouped by language.

also exhibit many control-flow-related bugs and higher patch coverage. Yet Python’s median TTD is 26.8 days, showing that the detection process takes considerably longer than in other languages. This suggests that language-specific factors might influence detection speed, and future studies should investigate these differences.

A Kruskal–Wallis test confirmed that these differences across languages are statistically significant ($H = 183.3, p < 0.01$), indicating that the time-to-detection of fuzzing bugs varies significantly by programming language. The post-hoc tests detected significant differences between all language pairs, except for Rust vs. Java.

Answer to RQ4: Fuzzing efficiency differs by programming language. Languages with higher patch coverage, like Go and Python, show longer detection times, while C, C++, Java, and Rust have lower coverage but faster detection. Though testing new code should enable early bug detection, this relationship does not hold consistently for fuzzing across languages.

5 Discussion

5.1 Lessons Learned

The central finding of this comprehensive empirical study is the conclusive confirmation that projects written in different programming languages have different characteristics and handling of fuzzing bugs. This section summarizes the resulting language-specific lessons learned, highlighting the distinct patterns for each language.

C and C++ projects show moderate to slightly higher median bug detection rates (0.02–0.025 issues per build), with C++ exhibiting the highest variability. Crash types are heavily dominated by Resource Management bugs (48.6% for C, 50.7% for C++), reflecting manual memory handling requirements. Both languages maintain low median patch coverage (~10%) but achieve quick detection times, with median TTD of 7.0 days for C and 9.5 days for C++.

Go projects face the greatest reproducibility challenge, with a median of only 50%. However, Go achieves the highest median patch coverage at around 30.2%.

Java projects present a unique security profile: the highest vulnerability ratio, yet nearly 100% are medium-severity (S3) due to memory management. Java achieves the fastest detection time with a median TTD of 2.8 days.

Python projects exhibit the lowest median bug detection rate (0.014) and its narrowest distribution. Control Flow bugs dominate at 79.0%. Despite higher patch coverage (median around 22.9%), Python has the longest median TTD at 26.8 days, indicating that coverage does not guarantee rapid detection.

Rust projects have a relatively high fuzzing bug detection rate but a significantly low vulnerability ratio (below 10%). Rust demonstrates superior reproducibility with the highest median reproducible ratio (around 98.8%). It achieves quick detection with a median TTD of 3.9 days, similar to Java.

Consideration of Confounding Factors. While our results exhibit clear differences across programming languages, the differences noted might not solely be attributable to language-specific problems. We utilized median values to help mitigate statistical bias introduced by individual project outliers and confounding factors. However, other factors such as project characteristics, might also impact our results and our observations may not stem exclusively from the programming language itself. It is important to acknowledge that all projects studied are well-established and selected by OSS-Fuzz. For instance, the Python projects analyzed are large with a median number of lines of code of 354,370 (See Table 1). Future research should identify the biggest contributing factors underlying these observations, using established techniques such as Regression Discontinuity Design (RDD) [12, 36, 37].

5.2 Implications

5.2.1 Implications for Researchers. Prior fuzzing research has predominantly focused on C or C++ projects, overlooking the fundamental impact of language differences. Since our comprehensive cross-language analysis demonstrates that programming languages fundamentally shape bug characteristics and detection efficiency, future studies must focus on language-specific bugs. This approach is necessary because different languages exhibit varying degrees of vulnerability resilience and severity, making it dangerous to generalize findings across languages. For instance, our results (RQ2) showed that C and C++ primarily suffer from Resource Management bugs, whereas Python and Rust tend to exhibit Control Flow bugs. Researchers should also pay attention to the impact of low bug detection rates. For example, if we look at Python projects, our results showed that Python projects tend to have a low detection rate (median value of 0.014 issues per build) but also that a significant proportion of fuzzing bugs (79.0%) found in these projects were Control flow Bugs. *Researchers should therefore be careful about whether the high percentage of Control Flow bugs reflects a genuinely high bug rate for that specific type bugs, or whether it indicates that current fuzzing techniques are ineffective at detecting other types of defects in Python.*

5.2.2 Implications for Fuzzing Platform Providers. Our findings highlight the need for language-specific fuzzers, given that fuzzing sessions and detected bugs manifest differently across programming languages. *Fuzzing platform providers should therefore offer specialized fuzzers for each supported language.* For example, fuzzers targeting C and C++ should prioritize detecting Resource Management bugs, which constitute nearly half of their vulnerabilities (48.6% and 50.7%, respectively). Conversely, fuzzers for interpreted languages like Python should be optimized to find Control Flow

issues. Several studies have developed fuzzers for Rust [14, 30], Python [20, 39], and Java [5, 31], which serve as viable alternatives.

Furthermore, given that patch coverage remains below 50% across most languages (RQ4), especially in C, C++, Java, and Rust (which all have medians below 10%), traditional fuzzing approaches using general Greybox Fuzzing Tools face significant limitations. *Fuzzing providers should offer Directed Greybox Fuzzing (DGF)[2] as an option.* DGF can focus on specific areas of code and can be directed to the changed lines of code in commits. The low coverage metrics indicate that general coverage-guided fuzzing (CGF) may fail to adequately test newly modified code, necessitating the use of directed approaches.

5.2.3 Implications for Practitioners. As observed, Java projects present a unique security profile: they have the highest overall vulnerability ratio among the languages studied, yet nearly 100% of these detected issues are classified as S3 (medium severity). This pattern suggests that Java’s built-in safety features, such as garbage collection and memory management, effectively prevent the most critical vulnerabilities common in other environments. Conversely, practitioners working with systems and scripting languages must prepare for inherently more severe vulnerabilities. C, C++, Go, and Rust primarily yield S1 (critical) and S2 (high severity) vulnerabilities. Python is particularly noteworthy, showing an exceptionally high proportion of S1 vulnerabilities, representing approximately 75% of its detected defects. These differences extend to crash types as well, where C and C++ primarily suffer from Resource Management bugs, while Java, Python, and Rust tend to show more Control Flow bugs.

Additionally, the language choice may dictate the efficiency of the bug remediation process, which is heavily influenced by reproducibility. Reproducing bugs consistently is essential for developers to successfully fix issues and confirm that the fixes are correct; conversely, unreproducible bugs waste development time and are less likely to be addressed. Our results revealed significant, language-dependent variations in the ratio of defects that were reproducible. Therefore, *practitioners should choose languages depending on their project’s security requirements, tolerance for critical vulnerabilities, and capacity for debugging reproducibility challenges.*

6 Threats to Validity

Threats to internal validity: Internal validity refers to the degree to which causal relationships can be established between variables without interference from confounding factors. In our study, internal validity is threatened by selection bias in the data collection process. We selected only projects where at least 10 fuzzing bugs were detected, which may introduce bias, as variations in detection rates could be attributed to project characteristics (e.g., projects with a tendency to frequently reveal bugs) rather than the factors we aim to investigate.

Threats to construct validity: Construct validity concerns whether the operational measures used in the study accurately represent the theoretical concepts being investigated. In our study, construct validity is threatened by potential confounding factors, as project maturity or configuration could influence bug detection rates rather

than the inherent properties of the programming language. To mitigate this threat, we calculated and compared the median values for metrics across projects, minimizing the statistical bias introduced by individual project outliers.

Threats to external validity: External validity addresses the extent to which the study’s findings can be generalized to other settings, populations, or conditions beyond the specific context of the research. In our study, external validity is threatened by the limited scope of our study population. This study is limited to projects that are continuously fuzzed by OSS-Fuzz, a service provided by Google. While OSS-Fuzz is the most widely used continuous fuzzing service today, it remains unclear whether our results can be generalized to other fuzzing tools, projects not participating in OSS-Fuzz, or commercial software development that does not incorporate continuous fuzzing into their CI/CD pipelines. Lastly, we applied strict filtering to obtain reliable median values. Although this smaller sample size may limit generalizability, it protects the robustness of our results against statistical noise.

7 Conclusion

This empirical study performed a comprehensive cross-language analysis to address the critical gap in previous language-agnostic fuzzing research, investigating how bug characteristics and detection efficiency vary across programming languages, utilizing 61,444 fuzzing bugs from 559 OSS-Fuzz projects.

Our findings confirm that programming language significantly influences the characteristics and handling of detected bugs, supporting the necessity of language-aware security analysis. While the median bug detection frequency (RQ1) and the fuzzing efficiency (Time-to-Detection, RQ4) remained generally consistent across languages—with median detection rates around 0.02 issues per build and TTDs between 5 and 25 days—the nature of the vulnerabilities differed sharply. Specifically, Java projects exhibited a distinct profile dominated by medium-severity (S3) vulnerabilities, contrasting with the predominance of critical and high-severity (S1/S2) vulnerabilities in C, C++, Go, Python, and Rust.

Furthermore, language design impacted reproducibility (RQ3): Rust projects demonstrated superior build reproducibility with the high median reproducible ratio (around 95%), while Go showed the poorest performance with a median near 50%. These results demonstrate that language-specific safety features correlate with distinct security outcomes in the OSS ecosystem.

Our future work should identify the biggest contributing factors underlying these observations, potentially utilizing established techniques such as Regression Discontinuity Design (RDD). We must also focus on language-specific bugs, investigating, for instance, whether the high percentage of Control Flow bugs in Python reflects a genuinely high bug rate for that specific type, or if it indicates that current fuzzing techniques are ineffective at detecting other types of defects in Python.

Acknowledgments

We gratefully acknowledge the financial support of JSPS for the KAKENHI grants (JP24K02921, JP24K02923, JP25K21359), as well as JST for the PRESTO grant (JPMJPR22P3), the ASPIRE grant (JPMJAP2415), and the AIP Accelerated Program (JPMJCR25U7).

References

- [1] Marcel Böhme and Brandon Falk. 2020. Fuzzing: on the exponential cost of vulnerability discovery. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (Virtual Event, USA) (ESEC/FSE'20). ACM, 713–724. <https://doi.org/10.1145/3368089.3409729>
- [2] Marcel Böhme, Van-Thuan Pham, Manh-Dung Nguyen, and Abhik Roychoudhury. 2017. Directed Greybox Fuzzing. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security* (Dallas, Texas, USA) (CCS '17). ACM, 2329–2344. <https://doi.org/10.1145/3133956.3134020>
- [3] Marcel Böhme, László Szekeres, and Jonathan Metzman. 2022. On the Reliability of Coverage-Based Fuzzer Benchmarking. In *Proceedings of the 44th International Conference on Software Engineering* (Pittsburgh, Pennsylvania, USA) (ICSE '22). ACM, 1621–1633. <https://doi.org/10.1145/3510003.3510230>
- [4] Sadullah Canakci, Nikolay Matyunin, Kalman Graffi, Ajay Joshi, and Manuel Egele. 2022. TargetFuzz: Using DARTs to Guide Directed Greybox Fuzzers. In *Proceedings of the 2022 ACM on Asia Conference on Computer and Communications Security* (Nagasaki, Japan) (ASIA CCS '22). ACM, 561–573. <https://doi.org/10.1145/3488932.3501276>
- [5] Yuting Chen, Ting Su, Chengnian Sun, Zhendong Su, and Jianjun Zhao. 2016. Coverage-directed differential testing of JVM implementations. In *Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Santa Barbara, CA, USA) (PLDI '16). ACM, 85–99. <https://doi.org/10.1145/2908080.2908095>
- [6] Lei Cui, Jiancong Cui, Zhiyu Hao, Lun Li, Zhenquan Ding, and Yongji Liu. 2022. An empirical study of vulnerability discovery methods over the past ten years. *Comput. Secur.* 120, C (Sept. 2022), 13 pages. <https://doi.org/10.1016/j.cose.2022.102817>
- [7] Zhen Yu Ding and Claire Le Goues. 2021. An Empirical Study of OSS-Fuzz Bugs. In *Proceedings of the 2021 IEEE/ACM 18th International Conference on Mining Software Repositories* (Madrid, Spain) (MSR'21). IEEE, 131–142. <https://doi.org/10.1109/MSR52588.2021.00026>
- [8] Google. 2025. *Fixing a bug | ClusterFuzz*. Retrieved October 24, 2025 from <https://google.github.io/clusterfuzz/using-clusterfuzz/workflows/fixing-a-bug/>
- [9] Google. 2025. *Fuzz Introspector*. Retrieved October 24, 2025 from <https://google.github.io/oss-fuzz/advanced-topics/fuzz-introspector/>
- [10] Keno Hassler, Philipp Görz, Stephan Lipp, Thorsten Holz, and Marcel Böhme. 2025. A Comparative Study of Fuzzers and Static Analysis Tools for Finding Memory Unsafety in C and C++. <https://doi.org/10.48550/arXiv.2505.22052>
- [11] Sture Holm. 1979. A Simple Sequentially Rejective Multiple Test Procedure. *Scandinavian Journal of Statistics* 6, 2 (Jan. 1979), 65–70. <https://doi.org/10.2307/4615733>
- [12] Guido Wilhelm. Imbens and Thomas Lemieux. 2008. Regression discontinuity designs: A guide to practice. *Journal of Econometrics* 142, 2 (Feb. 2008), 615–635. <https://doi.org/10.1016/j.jeconom.2007.05.001>
- [13] Nasif Intiaz and Laurie A. Williams. 2021. Memory Error Detection in Security Testing. <https://doi.org/10.48550/arXiv.2104.04385>
- [14] Jianfeng Jiang, Hui Xu, and Yangfan Zhou. 2021. RULF: Rust Library Fuzzing via API Dependency Graph Traversal. In *Proceedings of the 2021 36th IEEE/ACM International Conference on Automated Software Engineering* (Melbourne, Australia) (ASE'21). IEEE, 581–592. <https://doi.org/10.1109/ASE51524.2021.9678813>
- [15] Brandon N. Keller, Benjamin S. Meyers, and Andrew Meneely. 2023. What Happens When We Fuzz? Investigating OSS-Fuzz Bug History. In *Proceedings of the 2023 IEEE/ACM 20th International Conference on Mining Software Repositories* (Melbourne, Australia) (MSR'23). IEEE, 207–217. <https://doi.org/10.1109/MSR59073.2023.00038>
- [16] George Klees, Andrew Ruef, Benji Cooper, Shiyi Wei, and Michael Hicks. 2018. Evaluating Fuzz Testing. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security* (Toronto, Canada) (CCS '18). ACM, 2123–2138. <https://doi.org/10.1145/3243734.3243804>
- [17] Thijs Klooster, Fatih Turkmen, Gerben Broenink, Ruben ten Hove, and Marcel Böhme. 2023. Continuous Fuzzing: A Study of the Effectiveness and Scalability of Fuzzing in CI/CD Pipelines. In *Proceedings of the 2023 IEEE/ACM International Workshop on Search-Based and Fuzz Testing* (Melbourne, Australia) (SBFT'23). IEEE, 25–32. <https://doi.org/10.1109/SBFT59156.2023.00015>
- [18] Pavneet Singh Kochhar, Dinusha Wijedasa, and David Lo. 2016. A Large Scale Study of Multiple Programming Languages and Code Quality. In *Proceedings of the 2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER)* (Suita, Osaka, Japan) (SANER'16). IEEE, 563–573. <https://doi.org/10.1109/SANER.2016.112>
- [19] William Henry Kruskal and Wilson Allen Wallis. 1952. Use of Ranks in One-Criterion Variance Analysis. *J. Amer. Statist. Assoc.* 47, 260 (Dec. 1952), 583–621. <https://doi.org/10.1080/01621459.1952.10483441>
- [20] Wen Li, Haoran Yang, Xiapu Luo, Long Cheng, and Haipeng Cai. 2023. PyRTFuzz: Detecting Bugs in Python Runtimes via Two-Level Collaborative Fuzzing. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security* (Copenhagen, Denmark) (CCS '23). ACM, 1645–1659. <https://doi.org/10.1145/3576915.3623166>
- [21] Danushka Liyanage, Marcel Böhme, Chakkrit Tantithamthavorn, and Stephan Lipp. 2023. Reachable Coverage: Estimating Saturation in Fuzzing. In *Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering* (Melbourne, Australia) (ICSE'23). IEEE, 371–383. <https://doi.org/10.1109/ICSE48619.2023.00042>
- [22] Henry Berthold Mann and Donald Ransom Whitney. 1947. On a Test of Whether one of Two Random Variables is Stochastically Larger than the Other. *The Annals of Mathematical Statistics* 18, 1 (March 1947), 50–60. <https://doi.org/10.1214/aoms/1177730491>
- [23] Sebastian Nanz and Carlo A. Furia. 2015. A Comparative Study of Programming Languages in Rosetta Code. In *Proceedings of the 2015 IEEE/ACM 37th International Conference on Software Engineering* (Florence, Italy) (ICSE'15). IEEE, 778–788. <https://doi.org/10.1109/ICSE.2015.90>
- [24] Georgios Nikitopoulos, Konstantina Dritsa, Panos Louridas, and Dimitris Mitropoulos. 2021. CrossVul: a cross-language vulnerability dataset with commit data. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (Athens, Greece) (ESEC/FSE'21). ACM, 1565–1569. <https://doi.org/10.1145/3468264.3473122>
- [25] Olivier Nourry, Yutaro Kashiwa, Bin Lin, Gabriele Bavota, Michele Lanza, and Yasutaka Kamei. 2024. The Human Side of Fuzzing: Challenges Faced by Developers during Fuzzing Activities. *ACM Trans. Softw. Eng. Methodol.* 33, 1, Article 14 (Nov. 2024), 26 pages. <https://doi.org/10.1145/3611668>
- [26] Olivier Nourry, Yutaro Kashiwa, Weiyei Shang, Honglin Shu, and Yasutaka Kamei. 2025. My Fuzzers Won't Build: An Empirical Study of Fuzzing Build Failures. *ACM Trans. Softw. Eng. Methodol.* 34, 2, Article 29 (Jan. 2025), 30 pages. <https://doi.org/10.1145/3688842>
- [27] Olivier Nourry, Masanari Kondo, Mahmoud Alfadel, Shane McIntosh, and Yasutaka Kamei. 2024. Exploring the Adoption of Fuzz Testing in Open-Source Software: A Case Study of the Go Community. In *Proceedings of the 2024 IEEE International Conference on Software Maintenance and Evolution* (Flagstaff, AZ, USA) (ICSME'24). IEEE, 300–311. <https://doi.org/10.1109/ICSME58944.2024.00036>
- [28] Wonseok Oh and Hakjoo Oh. 2022. PyTER: effective program repair for Python type errors. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (Singapore, Singapore) (ESEC/FSE'22). ACM, 922–934. <https://doi.org/10.1145/3540250.3549130>
- [29] Oracle Corporation. 2023. *Secure Coding Guidelines for Java SE*. Retrieved October 24, 2025 from <https://www.oracle.com/java/technologies/javase/seccodguide.html>
- [30] David Paaßen, Jens-Rene Giesen, and Lucas Davi. 2025. Targeted Fuzzing for Unsafe Rust Code: Leveraging Selective Instrumentation. <https://doi.org/10.48550/arXiv.2505.02464>
- [31] Rohan Padhye, Caroline Lemieux, and Koushik Sen. 2019. JQF: coverage-guided property-based testing in Java. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis* (Beijing, China) (ISSTA'19). ACM, 398–401. <https://doi.org/10.1145/3293882.3339002>
- [32] Baishakhi Ray, Daryl Posnett, Premkumar Devanbu, and Vladimir Filkov. 2017. A large-scale study of programming languages and code quality in GitHub. *Commun. ACM* 60, 10 (Sept. 2017), 91–100. <https://doi.org/10.1145/3126905>
- [33] Tatsuya Shirai, Olivier Nourry, Yutaro Kashiwa, Kenji Fujiwara, Yasutaka Kamei, and Hajimu Iida. 2025. Large-Scale Empirical Analysis of Continuous Fuzzing: Insights from 1 Million Fuzzing Sessions. <https://doi.org/10.48550/ARXIV.2510.16433>
- [34] Diomidis Spinellis, Vassilios Karakoidas, and Panos Louridas. 2012. Comparative language fuzz testing: programming languages vs. fat fingers. In *Proceedings of the ACM 4th Annual Workshop on Evaluation and Usability of Programming Languages and Tools* (Tucson, Arizona, USA) (PLATEAU '12). ACM, 25–34. <https://doi.org/10.1145/2414721.2414727>
- [35] The MITRE Corporation. 2025. *Common Weakness Enumeration (CWE)*. Retrieved October 24, 2025 from <https://cwe.mitre.org/index.html>
- [36] Donald Knight Thistlethwaite and Donald Thomas Campbell. 1960. Regression-discontinuity analysis: An alternative to the ex post facto experiment. *Journal of Educational Psychology* 51, 6 (Dec. 1960), 309–317. <https://doi.org/10.1037/h0044319>
- [37] Mairieli Santos Wessel, Alexander Serebrenik, Igor Wiese, Igor Steinmacher, and Marco Aurélio Gerosa. 2020. Effects of Adopting Code Review Bots on Pull Requests to OSS Projects. In *Proceedings of the 2020 IEEE International Conference on Software Maintenance and Evolution* (Adelaide, Australia) (ICSME'20). IEEE, 1–11. <https://doi.org/10.1109/ICSME46990.2020.00011>
- [38] Yi Xiang, Xuhong Zhang, Peiyu Liu, Shouling Ji, Xiao Xiao, Hong Liang, Jiacheng Xu, and Wenhai Wang. 2024. Critical Code Guided Directed Greybox Fuzzing for Commits. In *Proceedings of the 33rd USENIX Security Symposium* (Philadelphia, PA, USA) (SEC '24). USENIX Association, Article 138, 16 pages. <https://www.usenix.org/conference/usenixsecurity24/presentation/xiang-yi>
- [39] Ilya Yegorov, Eli Kobrin, Darya Parygina, Alexey V. Vishnyakov, and Andrey Fedotov. 2024. Python Fuzzing for Trustworthy Machine Learning Frameworks.

- <https://doi.org/10.48550/arXiv.2403.12723> arXiv:2403.12723
- [40] Necip Fazıl Yıldıran, Jeho Oh, Julia Lawall, and Paul Gazzillo. 2024. Maximizing Patch Coverage for Testing of Highly-Configurable Software without Exploding Build Times. *Proc. ACM Softw. Eng.* 1, FSE, Article 20 (July 2024), 23 pages. <https://doi.org/10.1145/3643746>
- [41] Chengquan Zhang, Yang Feng, Yaokun Zhang, Yuxuan Dai, and Baowen Xu. 2024. Beyond Memory Safety: an Empirical Study on Bugs and Fixes of Rust Programs. In *Proceedings of the 2024 IEEE 24th International Conference on Software Quality, Reliability and Security* (Cambridge, United Kingdom) (QRS'24). IEEE, 272–283. <https://doi.org/10.1109/QRS62785.2024.00035>
- [42] Jiaming Zhang, Zhanqi Cui, Xiang Chen, Huiwen Yang, Liwei Zheng, and Jianbin Liu. 2023. CIDFuzz: Fuzz Testing for Continuous Integration. *IET Software* 17, 3 (June 2023), 301–315. <https://doi.org/10.1049/sfw2.12125>
- [43] Xiaogang Zhu and Marcel Böhme. 2021. Regression Greybox Fuzzing. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security* (Virtual Event, Republic of Korea) (CCS '21). ACM, 2169–2182. <https://doi.org/10.1145/3460120.3484596>